The Receipt That Never Ends 🧻 note.com Genre: Absurdist incremental clicker / management game Core idea: You start with a tiny receipt printer at a boring checkout counter. You earn money by printing receipts. Soon, you discover that the longer your receipts get, the more valuable they b

pasted

2) Implementing Requirements This table should serve as a draft for how you will implement the requirements into your final project. You may change your ideas later on, but your final site must fulfill all the requirements. Requirement Implementation Purpose/Functionality A 1-2 sentence

pasted

create me a website using drafter with best code practices to meet these requirements Below are the requirements that your site must fulfill. Plan ahead so that meeting these requirements is easier as you develop your project.

- At least 7 routes.
- At least 5 Pages, or 10 FragmentsLinks to an external site., or a combination thereof.
- The implementation of the State dataclass must have at least 4 fields.
    - At least one field must be a list of class instances or a dictionary of class instances.
        - This means that there must be a second dataclass.
    - At least 4 state fields, including the list or dictionary of class instances, must each be meaningfully modified in at least one route; those 4 state fields cannot be constants.
- At least 3 input fields (TextBox, CheckBox, SelectBox, TextArea). They don't have to be the same type.
- At least 3 meaningful if statements.
- At least 1 loop that iterates over an attribute of the list of class instances or dictionary of class instances in a meaningful way.
- The site should have a legitimate purpose or functionality.
- No global variables (global constantsLinks to an external site. are fine).
- You must use the Drafter library.
- Any advanced coding techniques (i.e. that weren't covered in this course) used are thoroughly documented in the code.

More creative and impressive designs are given flexibility on the above requirements - impress us! However, impressive websites tend to do more than the above requirements. Further, if you miss more than 2 requirements, you will get no credit for that submission. For example, missing a second dataclass means you would be missing not only that requirement, but also the list or dictionary of such dataclass instances, and the loop

iterating over that list. This would mean that the submission would get a zero.Make sure your program and website runs; broken programs do not earn points. This means that if your program contains a syntax error or runs and doesn't deploy a website, you get zero points for this assignment, and are subject to any other penalties listed in the syllabus. Once you are finished with your project code, check over the following:

- Chosen good variable names (check last two sections of Chapter 1B1Links to an external site.)
- Unit tested your functions where possible, being valid and thoroughLinks to an external site.: each route must have at least one unit test, if possible (routes that have randomization are difficult to unit test, so they are not required to).
    - If a function can't be unit tested, you must note as much in a docstring or comment.
- Broke up excessively long lines (80 characters is a good rule of thumb, but we'll be strict about not exceeding 120 characters)
- Included types in your function headers
- Included good docstrings Links to an external site.as the first line of your function's body
- Decomposed complicated functions with helper functions

I built the game as a single file, `receipt_game.py`. I couldn't run it in real Drafter here, so I checked the logic against a stand-in for the library. All the unit tests pass and no line goes over 100 characters. Run it in your Drafter setup before you submit.

Requirements

Requirement	How it's met
7+ routes	12: `index`, `print_receipt`, `shop_page`, `buy_upgrade`, `stats_page`, `receipt_history`, `settings_page`, `save_settings`, `info_page`, `reset_page`, `game_reset`, `victory_page`
5+ pages	8 pages: factory, shop, stats, receipt history, settings, info, reset confirmation, victory
State with 4+ fields	10 fields, including `upgrades: list[Upgrade]`
Second dataclass	`Upgrade` (name, description, base cost, click bonus, times bought, purchase memo)

4+ fields modified in routes	`money`, `total_earned`, `total_clicks`, `money_per_click`, `company_name`, `department`, `show_milestones`, `has_won`, and `Upgrade.times_bought`
3+ inputs	`TextBox` for the company name, `SelectBox` for the department, `CheckBox` for milestone memos (all on the settings page)
3+ meaningful ifs	Can the player afford the upgrade, has the player won, which milestone memo applies, is the department unlocked yet, empty-name fallback
Loop over the list of instances	`build_shop_rows`, `build_receipt_lines`, `count_upgrades_bought`, `find_upgrade_index`
No global variables	Only UPPER_CASE constants, and the shop is created fresh by `make_upgrades()`
Advanced techniques documented	The module docstring explains `f"{x:,}"`, `Argument(...)`, and `250_000`
Unit tests	Every route has a test, and so does every helper. None use randomness
Style	Type hints and docstrings on every function, long logic split into small helpers

Differences from your planning table

- `Upgrade` has `times_bought: int` instead of `bought: bool`. Upgrades can be bought repeatedly, each copy costing 15% more, which gives the game more to do.
- The department SelectBox is locked behind lifetime earnings (Big Business at $5,000, Bigger Business at $50,000). That makes it a real if-statement decision.
- The win condition is $250,000 in lifetime earnings, set by the `WIN_TARGET` constant. After winning, “Keep playing” returns to the factory page for endless mode.
- I skipped passive income and timers, as your planning doc suggested.

Drafter calls to check

- I'm confident about the core API (`@route`, `Page`, `Button`, `Link`, `Header`, `TextBox`, `CheckBox`, `SelectBox`, `BulletedList`, `start_server`) and `Argument`, but your course version may differ slightly.
- The styling calls `add_website_css` and `set_website_title` are the likeliest to differ. They're isolated in `style_website()`, so if they error, delete that one line near the bottom.
- If your course requires `set_site_information(...)`, add it next to `style_website()`.
- I used `index` as the factory page, since Drafter treats the route named `index` as the home page.

Before you submit

- Click through every page, buy an upgrade, change settings, and test reset.
- To see the win screen quickly, temporarily lower `WIN_TARGET`.

Receipt game

PY

turn these into buttons Factory Shop Receipt History Stats Settings How to Play, use assert equal tests, add a : after LATEST COMPANY MEMO, and remove YOUR COMPANY BALANCE

I made all four changes in `receipt_game.py`:

- Nav buttons: the six nav links in `nav_bar()` (Factory, Shop, Receipt History, Stats, Settings, How to Play) are now `Button`s that point at the route functions.
- Unit tests: every check in the five test functions now uses `assert_equal(actual, expected)`. Yes/no checks compare against `True` or `False`, and the `in` checks do too.
- Memo label: the factory page now says "LATEST COMPANY MEMO:".
- Balance label: "YOUR COMPANY BALANCE" is gone. The big money number now sits right under the rank line.

I couldn't run it in real Drafter here. I ran it against a stand-in for the library, and the tests pass and no line is over 100 characters.

Since the nav is buttons now, check that each one is clickable on every page in your Drafter setup.

Receipt game

PY"""

Definitely Necessary Receipts, Inc.

An absurdist clicker game built with Drafter. The player prints receipts to
earn money, buys increasingly ridiculous upgrades, and tries to print
"The Receipt for Everything" by earning WIN_TARGET dollars in total.

Pages: index (factory), shop_page, stats_page, receipt_history, settings_page,
        info_page, reset_page, victory_page
Routes: all of the pages above plus print_receipt, buy_upgrade,
        save_settings, and game_reset

Design notes
----
* The game state lives in the State dataclass (no global variables). The only
  module-level names are UPPER_CASE constants that never change.
* The upgrade shop is a list of Upgrade dataclass instances stored in the
  state, so one loop can draw every upgrade without repeating code.
* No advanced techniques are used beyond what the course covered, except:
  - f-strings with a format spec, e.g. f"{amount:,}", which add commas to a
    number (12345 becomes "12,345").
  - Argument(...), a Drafter feature that lets a Button send one value
    (here, the upgrade's name) to the route it links to.
  - Numbers written like 250_000: Python lets you use underscores inside a
    number to make it easier to read. 250_000 is exactly 250000.
"""
from dataclasses import dataclass

from drafter import *

```
# Constants (allowed: they are never changed while the program runs)
# ----------------------------------------------------------------------
DEFAULT_COMPANY_NAME = "Definitely Necessary Receipts, Inc."
MAX_NAME_LENGTH = 40
STARTING_CLICK_VALUE = 1
PRICE_GROWTH_RATE = 1.15  # each copy of an upgrade costs 15% more than the last
WIN_TARGET = 250_000  # lifetime dollars needed to print the Receipt for Everything
PROGRESS_BAR_WIDTH = 20
STARTING_MEMO = "Welcome. Please print receipts. Do not ask why."

# Total money a player must have earned before a department can be chosen.
DEPARTMENT_UNLOCKS = {
    "Business": 0,
    "Big Business": 5_000,
    "Bigger Business": 50_000,
}

# Tables of (minimum lifetime earnings, text). Listed from highest to lowest
# so that the first row that fits is the best one the player has reached.
RANK_TITLES = [
    (100_000, "Supreme Receipt Officer"),
    (25_000, "CEO of Nothing"),
    (1_000, "Paper Pusher"),
    (0, "Humble Cashier"),
]
MILESTONE_MEMOS = [
    (100_000, "Management approved receipts for future purchases. "
              "Legal says this is probably fine."),
    (10_000, "The accounting department has invoiced itself. The company is "
             "now its own best customer and its own worst customer."),
```

```
    (1_000, "The intern printed 400 receipts during lunch. Productivity is "
            "up 900%. Nobody knows what productivity means."),
    (100, "Congratulations! You have earned your first $100. "
          "Nobody asked for it."),
    (0, STARTING_MEMO),
]



# ---------------------------------------------------------------------------
# Dataclasses
# ---------------------------------------------------------------------------
@dataclass
class Upgrade:
    """One item in the shop that permanently increases money per click."""
    name: str
    description: str
    base_cost: int  # price of the first copy; later copies cost more
    click_bonus: int  # extra dollars per click for each copy owned
    times_bought: int  # how many copies the player owns
    purchase_memo: str  # joke shown right after buying it



@dataclass
class State:
    """Everything the game needs to remember between page loads."""
    money: int  # spendable dollars
    total_earned: int  # lifetime dollars, never goes down when spending
    total_clicks: int  # receipts printed by clicking
    money_per_click: int  # dollars earned by each click
    company_name: str  # chosen by the player on the settings page
```

```
    department: str  # chosen by the player; locked behind earnings
    show_milestones: bool  # whether company memos are shown
    last_memo: str  # most recent message to display to the player
    has_won: bool  # True once the Receipt for Everything is printed
    upgrades: list[Upgrade]  # the shop, one Upgrade per item


# ---------------------------------------------------------------------
# Helper functions (pure logic, easy to unit test)
# ---------------------------------------------------------------------
def make_upgrades() -> list[Upgrade]:
    """Create a fresh shop where nothing has been bought yet."""
    return [
        Upgrade("Slightly Better Printer",
                "Prints 1% faster. Management rounds this up to a dollar.",
                10, 1, 0, "The printer has requested a paid vacation."),
        Upgrade("Suspiciously Long Paper",
                "Receipts are now longer than the purchases. Customers "
                "are concerned.",
                50, 5, 0, "The receipt is now taller than the cashier."),
        Upgrade("Unpaid Intern",
                "Prints receipts all day. Has politely requested a chair.",
                150, 10, 0, "The intern says 'exposure' is not a currency."),
        Upgrade("Printer Farm",
                "A whole field of printers. They need sunlight and toner.",
                500, 40, 0, "The printers have formed a union."),
        Upgrade("Accounting Department",
                "Charges you for the receipts it uses to track receipts.",
                2_000, 150, 0, "Accounting has invoiced accounting."),
        Upgrade("Time-Travel Receipt",
```

```
            "Collect payment for purchases that have not happened yet.",
            8_000, 600, 0, "Legal says this is probably fine."),
        Upgrade("Universal Paper Roll",
            "Enough paper to receipt the entire universe. Existing "
            "now carries a service fee.",
            30_000, 2_500, 0, "The universe has been notified of its fee."),
    ]


def make_new_state() -> State:
    """Create the state for a brand new game."""
    return State(0, 0, 0, STARTING_CLICK_VALUE, DEFAULT_COMPANY_NAME,
                 "Business", True, STARTING_MEMO, False, make_upgrades())


def format_money(amount: int) -> str:
    """Turn a whole number of dollars into text such as '$1,250'."""
    return f"${amount:,}"


def upgrade_price(upgrade: Upgrade) -> int:
    """Return the current price of an upgrade (rises with each copy owned)."""
    return round(upgrade.base_cost * PRICE_GROWTH_RATE ** upgrade.times_bought)


def can_afford(money: int, price: int) -> bool:
    """Return True if the player has at least `price` dollars."""
    return money >= price
```

```
def find_upgrade_index(upgrades: list[Upgrade], name: str) -> int:
    """Return the position of the upgrade called `name`, or -1 if missing."""
    for index_number, upgrade in enumerate(upgrades):
        if upgrade.name == name:
            return index_number
    return -1


def try_purchase(state: State, upgrade: Upgrade) -> bool:
    """Buy one copy of `upgrade` if affordable; return whether it worked."""
    price = upgrade_price(upgrade)
    if not can_afford(state.money, price):
        return False
    state.money -= price
    upgrade.times_bought += 1
    state.money_per_click += upgrade.click_bonus
    return True


def count_upgrades_bought(upgrades: list[Upgrade]) -> int:
    """Add up how many upgrades (counting every copy) have been bought."""
    total = 0
    for upgrade in upgrades:
        total += upgrade.times_bought
    return total


def pick_by_threshold(table: list[tuple[int, str]], amount: int) -> str:
    """Return the text from the first row whose threshold `amount` reaches.
```

```
    The table must be sorted from the highest threshold to the lowest.
    """
    for threshold, text in table:
        if amount >= threshold:
            return text
    return table[-1][1]


def get_rank_title(total_earned: int) -> str:
    """Return the player's job title based on lifetime earnings."""
    return pick_by_threshold(RANK_TITLES, total_earned)


def get_company_memo(total_earned: int, show_milestones: bool) -> str:
    """Return the milestone memo for the player's progress (or a notice)."""
    if not show_milestones:
        return "Company memos have been disabled by management."
    return pick_by_threshold(MILESTONE_MEMOS, total_earned)


def has_reached_goal(total_earned: int) -> bool:
    """Return True once lifetime earnings reach the winning amount."""
    return total_earned >= WIN_TARGET


def percent_to_goal(total_earned: int) -> int:
    """Return progress toward the win condition as a whole number 0-100."""
    return min(100, total_earned * 100 // WIN_TARGET)
```

```
def make_progress_bar(percent: int) -> str:
    """Draw a text progress bar such as '[####----------] 20%'."""
    filled = percent * PROGRESS_BAR_WIDTH // 100
    empty = PROGRESS_BAR_WIDTH - filled
    return "[" + "#" * filled + "-" * empty + f"] {percent}%"


def is_department_unlocked(department: str, total_earned: int) -> bool:
    """Return True if the player has earned enough to join `department`."""
    needed = DEPARTMENT_UNLOCKS.get(department, 0)
    return total_earned >= needed


def clean_company_name(raw_name: str) -> str:
    """Trim a typed company name; fall back to the default if it is blank."""
    trimmed = raw_name.strip()[:MAX_NAME_LENGTH]
    if trimmed == "":
        return DEFAULT_COMPANY_NAME
    return trimmed


def get_display_name(state: State) -> str:
    """Return the company name with its department, e.g. 'Acme (Big Business)'."""
    return f"{state.company_name} ({state.department} Division)"


# ---------------------------------------------------------------------
# Helper functions that build pieces of pages
# ---------------------------------------------------------------------
def nav_bar() -> list:
```

```
    """Return the navigation buttons shown at the top of every page."""
    return [
        Button("Factory", index),
        Button("Shop", shop_page),
        Button("Receipt History", receipt_history),
        Button("Stats", stats_page),
        Button("Settings", settings_page),
        Button("How to Play", info_page),
        HorizontalRule(),
    ]



def build_shop_rows(state: State) -> list:
    """Loop over every upgrade and build the shop entry for each one."""
    rows = []
    for upgrade in state.upgrades:
        price = upgrade_price(upgrade)
        rows.append(Header(f"{upgrade.name} (owned: {upgrade.times_bought})", 3))
        rows.append(upgrade.description)
        rows.append(f"Effect: +{format_money(upgrade.click_bonus)} per click")
        if can_afford(state.money, price):
            rows.append(Button(f"Buy for {format_money(price)}", buy_upgrade,
                               [Argument("upgrade_name", upgrade.name)]))
        else:
            missing = price - state.money
            rows.append(f"Costs {format_money(price)} "
                        f"(you need {format_money(missing)} more)")
        rows.append(HorizontalRule())
    return rows
```

```
def build_receipt_lines(upgrades: list[Upgrade]) -> list:
    """Loop over the upgrades and print one receipt line per owned upgrade."""
    lines = []
    for upgrade in upgrades:
        if upgrade.times_bought > 0:
            bonus = upgrade.times_bought * upgrade.click_bonus
            lines.append(f"{upgrade.times_bought} x {upgrade.name} "
                         f"... +{format_money(bonus)} per click")
    return lines



# ---------------------------------------------------------------------------
# Routes (pages and actions)
# ---------------------------------------------------------------------------
@route
def index(state: State) -> Page:
    """Show the factory page: balance, rank, memo, and the print button."""
    percent = percent_to_goal(state.total_earned)
    return Page(state, nav_bar() + [
        Header(get_display_name(state), 1),
        f"Rank: {get_rank_title(state.total_earned)}",
        Header(format_money(state.money), 2),
        f"Each receipt earns {format_money(state.money_per_click)}",
        Button("PRINT RECEIPT", print_receipt),
        HorizontalRule(),
        "LATEST COMPANY MEMO:",
        state.last_memo,
        HorizontalRule(),
        "Progress toward the Receipt for Everything:",
```

```
        make_progress_bar(percent),
    ])


@route
def print_receipt(state: State) -> Page:
    """Earn money for one click, update the memo, and check for victory."""
    state.money += state.money_per_click
    state.total_earned += state.money_per_click
    state.total_clicks += 1
    state.last_memo = get_company_memo(state.total_earned,
                                       state.show_milestones)
    if has_reached_goal(state.total_earned) and not state.has_won:
        state.has_won = True
        return victory_page(state)
    return index(state)


@route
def shop_page(state: State) -> Page:
    """Show every upgrade with its price and a Buy button when affordable."""
    return Page(state, nav_bar() + [
        Header("Upgrade Shop", 1),
        f"Balance: {format_money(state.money)}",
        state.last_memo,
        HorizontalRule(),
    ] + build_shop_rows(state))


@route
```

```
def buy_upgrade(state: State, upgrade_name: str) -> Page:
    """Buy the named upgrade if the player can afford it."""
    position = find_upgrade_index(state.upgrades, upgrade_name)
    if position == -1:
        state.last_memo = "That upgrade does not exist. Please file a form."
        return shop_page(state)
    upgrade = state.upgrades[position]
    if try_purchase(state, upgrade):
        state.last_memo = upgrade.purchase_memo
    else:
        short = upgrade_price(upgrade) - state.money
        state.last_memo = f"Insufficient funds: you need {format_money(short)} more."
    return shop_page(state)



@route
def stats_page(state: State) -> Page:
    """Show lifetime statistics and the progress toward the win condition."""
    status = "Universe receipted" if state.has_won else "Still printing"
    return Page(state, nav_bar() + [
        Header("Company Statistics", 1),
        f"Company: {get_display_name(state)}",
        f"Rank: {get_rank_title(state.total_earned)}",
        f"Lifetime earnings: {format_money(state.total_earned)}",
        f"Current balance: {format_money(state.money)}",
        f"Receipts printed: {state.total_clicks:,}",
        f"Money per click: {format_money(state.money_per_click)}",
        f"Upgrades purchased: {count_upgrades_bought(state.upgrades)}",
        f"Goal: {format_money(WIN_TARGET)}",
        make_progress_bar(percent_to_goal(state.total_earned)),
```

```
        f"Status: {status}",
        HorizontalRule(),
        Button("Reset game", reset_page),
    ])


@route
def receipt_history(state: State) -> Page:
    """Show the player's purchases formatted like a receipt."""
    lines = build_receipt_lines(state.upgrades)
    if len(lines) == 0:
        lines = ["No purchases yet. Suspicious."]
    return Page(state, nav_bar() + [
        Header("Receipt History", 1),
        "*** OFFICIAL RECEIPT ***",
        get_display_name(state),
        HorizontalRule(),
        BulletedList(lines),
        HorizontalRule(),
        f"TOTAL UPGRADES: {count_upgrades_bought(state.upgrades)}",
        "THANK YOU FOR YOUR EXISTENCE.",
    ])


@route
def settings_page(state: State) -> Page:
    """Show the three input fields: name, department, and memo checkbox."""
    return Page(state, nav_bar() + [
        Header("Company Settings", 1),
        state.last_memo,
```

```
        "Company name:",
        TextBox("company_name", state.company_name),
        "Department (higher departments unlock as you earn money):",
        SelectBox("department", list(DEPARTMENT_UNLOCKS), state.department),
        CheckBox("show_milestones", state.show_milestones),
        "Show company memos",
        Button("Save settings", save_settings),
    ])


@route
def save_settings(state: State, company_name: str, department: str,
                  show_milestones: bool) -> Page:
    """Save the typed name and checkbox; save the department if unlocked."""
    state.company_name = clean_company_name(company_name)
    state.show_milestones = show_milestones
    if is_department_unlocked(department, state.total_earned):
        state.department = department
        state.last_memo = "Settings saved. Management is mildly pleased."
    else:
        needed = DEPARTMENT_UNLOCKS.get(department, 0)
        state.last_memo = (f"{department} is locked until you have earned "
                           f"{format_money(needed)}.")
    return settings_page(state)


@route
def info_page(state: State) -> Page:
    """Explain how to play the game."""
    return Page(state, nav_bar() + [
```

```
        Header("How to Play", 1),
        "You work for a company whose only product is proof that the "
        "company exists. Your boss has one instruction: print receipts. "
        "Do not ask why.",
        BulletedList([
            "Press PRINT RECEIPT on the Factory page to earn money.",
            "Spend money in the Shop to earn more per click. Every copy "
            "of an upgrade costs 15% more than the last.",
            "Earn more to climb the ranks and unlock new departments in "
            "Settings.",
            f"Earn {format_money(WIN_TARGET)} in total to print the "
            "Receipt for Everything.",
        ]),
    ])


@route
def reset_page(state: State) -> Page:
    """Ask the player to confirm before erasing all progress."""
    return Page(state, nav_bar() + [
        Header("Reset the company?", 1),
        "This erases your money, upgrades, and statistics. "
        "Even the universe will forget you.",
        Button("Yes, reset everything", game_reset),
        Button("Cancel", index),
    ])


@route
def game_reset(state: State) -> Page:
```

```
    """Throw away the old state and start a brand new game."""
    return index(make_new_state())


@route
def victory_page(state: State) -> Page:
    """Show the final receipt after the player reaches the win condition."""
    return Page(state, [
        Header("TRANSACTION COMPLETE", 1),
        "Item: The Entire Universe",
        f"Subtotal: {format_money(WIN_TARGET - 1)}.99",
        "Administrative Fee: $0.01",
        f"TOTAL: {format_money(WIN_TARGET)}.00",
        HorizontalRule(),
        "THANK YOU FOR YOUR EXISTENCE.",
        "Please keep this receipt. It is required to prove that reality "
        "occurred. No refunds. The universe is final sale.",
        HorizontalRule(),
        Button("View final statistics", stats_page),
        Button("Keep playing (endless mode)", index),
        Button("Start over", game_reset),
    ])


# ---------------------------------------------------------------------
# Unit tests
# Routes are tested by calling them directly and checking the returned
# Page's state, using Drafter's assert_equal. Every route has at least one
# test. None of them use randomness, so all of them can be tested.
# ---------------------------------------------------------------------
```

```
def test_formatting_helpers() -> None:
    """Test format_money, make_progress_bar, and clean_company_name."""
    assert_equal(format_money(0), "$0")
    assert_equal(format_money(1250), "$1,250")
    assert_equal(format_money(1_000_000), "$1,000,000")
    assert_equal(make_progress_bar(0), "[" + "-" * 20 + "] 0%")
    assert_equal(make_progress_bar(50), "[" + "#" * 10 + "-" * 10 + "] 50%")
    assert_equal(make_progress_bar(100), "[" + "#" * 20 + "] 100%")
    assert_equal(clean_company_name("  Acme  "), "Acme")
    assert_equal(clean_company_name("   "), DEFAULT_COMPANY_NAME)
    assert_equal(len(clean_company_name("x" * 100)), MAX_NAME_LENGTH)


def test_upgrade_helpers() -> None:
    """Test upgrade_price, can_afford, find_upgrade_index, try_purchase."""
    sample = Upgrade("Test", "desc", 100, 5, 0, "memo")
    assert_equal(upgrade_price(sample), 100)
    sample.times_bought = 1
    assert_equal(upgrade_price(sample), 115)
    sample.times_bought = 2
    assert_equal(upgrade_price(sample), 132)
    assert_equal(can_afford(10, 10), True)
    assert_equal(can_afford(9, 10), False)
    shop = make_upgrades()
    assert_equal(find_upgrade_index(shop, "Unpaid Intern"), 2)
    assert_equal(find_upgrade_index(shop, "Nonexistent"), -1)
    state = make_new_state()
    assert_equal(try_purchase(state, state.upgrades[0]), False)
    state.money = 10
    assert_equal(try_purchase(state, state.upgrades[0]), True)
```

```
    assert_equal(state.money, 0)
    assert_equal(state.money_per_click, 2)
    assert_equal(state.upgrades[0].times_bought, 1)


def test_progress_helpers() -> None:
    """Test ranks, memos, goal checks, counting, and departments."""
    assert_equal(get_rank_title(0), "Humble Cashier")
    assert_equal(get_rank_title(1_000), "Paper Pusher")
    assert_equal(get_rank_title(500_000), "Supreme Receipt Officer")
    assert_equal(get_company_memo(0, True), STARTING_MEMO)
    assert_equal("first $100" in get_company_memo(150, True), True)
    assert_equal("disabled" in get_company_memo(150, False), True)
    assert_equal(has_reached_goal(WIN_TARGET), True)
    assert_equal(has_reached_goal(WIN_TARGET - 1), False)
    assert_equal(percent_to_goal(0), 0)
    assert_equal(percent_to_goal(WIN_TARGET * 2), 100)
    assert_equal(count_upgrades_bought(make_upgrades()), 0)
    owned = make_upgrades()
    owned[0].times_bought = 2
    owned[3].times_bought = 1
    assert_equal(count_upgrades_bought(owned), 3)
    assert_equal(build_receipt_lines(owned), [
        "2 x Slightly Better Printer ... +$2 per click",
        "1 x Printer Farm ... +$40 per click",
    ])
    assert_equal(is_department_unlocked("Business", 0), True)
    assert_equal(is_department_unlocked("Big Business", 4_999), False)
    assert_equal(is_department_unlocked("Big Business", 5_000), True)
```

```
def test_game_routes() -> None:
    """Test every route by calling it and checking the new state."""
    start = make_new_state()
    assert_equal(start.money, 0)
    assert_equal(start.has_won, False)
    assert_equal(index(start).state.money, 0)

    clicked = print_receipt(make_new_state()).state
    assert_equal(clicked.money, 1)
    assert_equal(clicked.total_earned, 1)
    assert_equal(clicked.total_clicks, 1)

    almost_won = make_new_state()
    almost_won.total_earned = WIN_TARGET - 1
    assert_equal(print_receipt(almost_won).state.has_won, True)

    assert_equal(shop_page(make_new_state()).state.money, 0)

    rich = make_new_state()
    rich.money = 10
    bought = buy_upgrade(rich, "Slightly Better Printer").state
    assert_equal(bought.money, 0)
    assert_equal(bought.money_per_click, 2)
    broke = buy_upgrade(make_new_state(), "Slightly Better Printer").state
    assert_equal(broke.money_per_click, 1)
    assert_equal("Insufficient" in broke.last_memo, True)
    missing = buy_upgrade(make_new_state(), "Nonexistent").state
    assert_equal(missing.money_per_click, 1)
```

```
    assert_equal(stats_page(make_new_state()).state.total_clicks, 0)
    assert_equal(receipt_history(make_new_state()).state.money, 0)
    assert_equal(info_page(make_new_state()).state.money, 0)
    assert_equal(reset_page(make_new_state()).state.money, 0)
    assert_equal(victory_page(make_new_state()).state.has_won, False)
    assert_equal(settings_page(make_new_state()).state.department, "Business")


def test_settings_and_reset_routes() -> None:
    """Test save_settings (unlocked and locked) and game_reset."""
    allowed = save_settings(make_new_state(), " Acme ", "Business", False).state
    assert_equal(allowed.company_name, "Acme")
    assert_equal(allowed.show_milestones, False)
    locked = save_settings(make_new_state(), "Acme", "Bigger Business", True).state
    assert_equal(locked.department, "Business")
    assert_equal("locked" in locked.last_memo, True)
    unlocked_state = make_new_state()
    unlocked_state.total_earned = 5_000
    chosen = save_settings(unlocked_state, "Acme", "Big Business", True).state
    assert_equal(chosen.department, "Big Business")
    dirty = make_new_state()
    dirty.money = 999
    dirty.upgrades[0].times_bought = 3
    fresh = game_reset(dirty).state
    assert_equal(fresh.money, 0)
    assert_equal(fresh.upgrades[0].times_bought, 0)


def run_unit_tests() -> None:
    """Run all unit tests; an AssertionError means something is broken."""
```

```
    test_formatting_helpers()
    test_upgrade_helpers()
    test_progress_helpers()
    test_game_routes()
    test_settings_and_reset_routes()


# ---------------------------------------------------------------------
# Start the website
# ---------------------------------------------------------------------
def style_website() -> None:
    """Give the site a receipt-paper look (beige paper, black ink, green)."""
    set_website_title("Definitely Necessary Receipts, Inc.")
    add_website_css("body", "background-color: #F5F0E6; color: #242424; "
                            "font-family: 'Courier New', monospace;")
    add_website_css("button", "background-color: #285943; color: #F5F0E6; "
                              "border: none; padding: 10px 18px; "
                              "font-family: 'Courier New', monospace; "
                              "cursor: pointer;")


run_unit_tests()
style_website()
start_server(make_new_state())
```